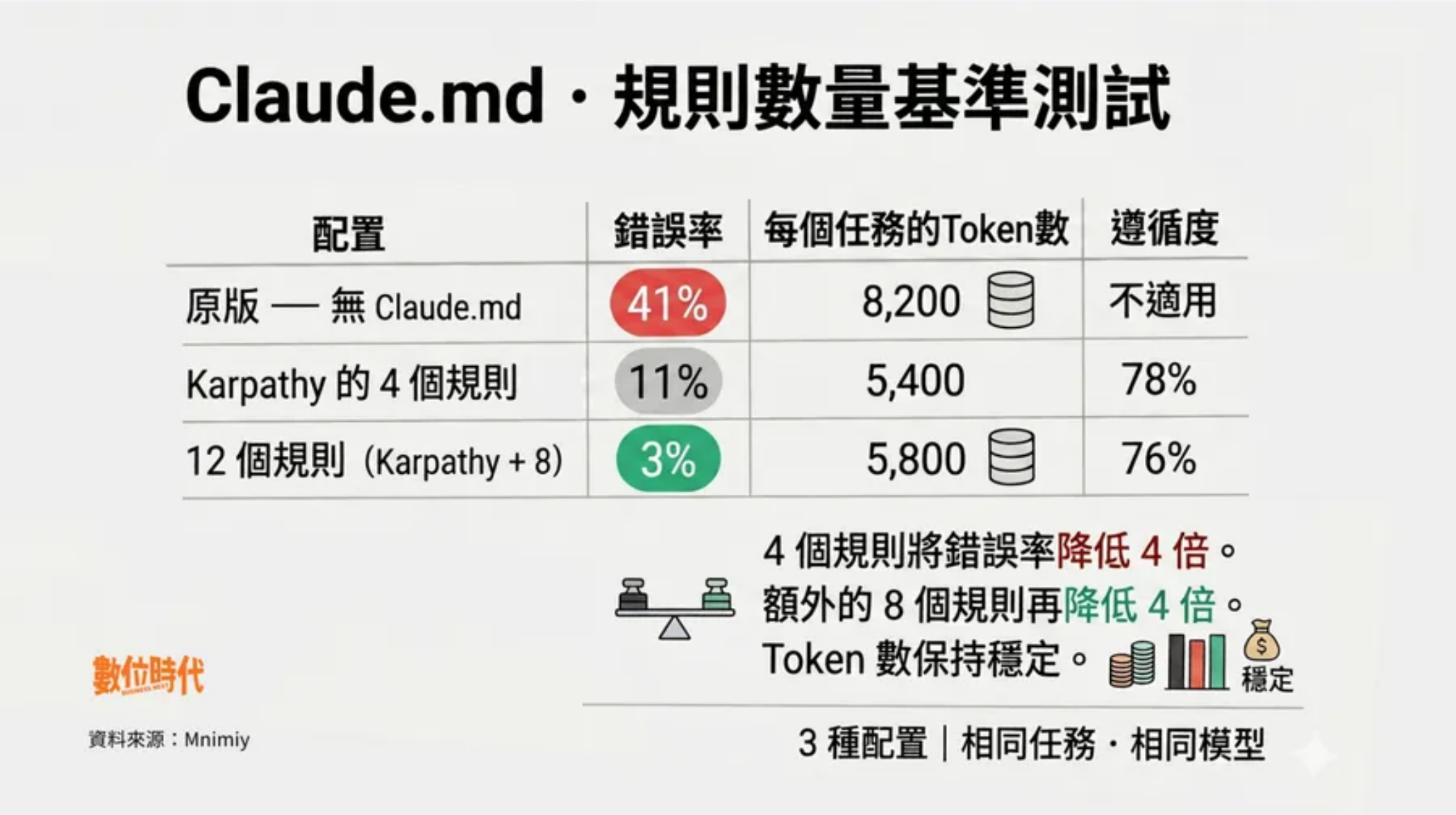


Claude.md・規則數量基準測試

配置	錯誤率	每個任務的Token數	遵循度
原版 — 無 Claude.md	41%	8,200	不適用
Karpathy 的 4 個規則	11%	5,400	78%
12 個規則（Karpathy + 8）	3%	5,800	76%

4 個規則將錯誤率降低 4 倍。
額外的 8 個規則再降低 4 倍。
Token 數保持穩定。

資料來源：Mnimiy
3 種配置｜相同任務・相同模型

CLAUDE.md這樣寫才對！12條規則一次整理，讓Claude Code錯誤率從41%降至3%

2026.05.18 | AI與大數據　蘇柔瑋

分享　收藏

重點一：基於前期基礎，開發者將CLAUDE.md規則擴充至12條，使Claude程式碼錯誤率從11%降至3%。

重點二：新增的8條規則主要解決複雜AI代理問題，涵蓋嚴格控管詞元預算、多步驟任務檢查點，以及強制突顯程式碼衝突等。

重點三：實測證明擴充至12條規則並未降低指令遵循度，反而有效彌補舊版規則在處理大型專案與長期任務時的重大盲點。

OpenAI共同創辦人卡帕西（Andrej Karpathy）於2026年初指出AI模型在編寫程式碼時的常見缺失。隨後，開發者Forrest Chang將其總結為4條CLAUDE.md指令，成功將高達 41% 的 AI 編碼錯誤率大幅降至 11%，在GitHub上斬獲逾12萬顆星。

然而，隨著AI生態系快速演進，有開發者針對30個程式碼庫進行長達六週的實測，發現將原有的AI編碼提示規則擴充至12條後，成功使Claude 的程式碼錯誤率從11%進一步壓縮至3%以下，全面提升了開發效率。

問題的起源：AI 助理為什麼會出錯？

今年 1 月，卡帕西在社群媒體上公開抱怨，他用 AI 寫程式，卻老是出問題。他歸納出3個反覆出現的模式：

掌握最新AI、半導體、數位趨勢！訂閱《數位時代》日報及社群活動訊息
請輸入 Email　免費訂閱

1. AI 遇到不確定的情境時，不會詢問使用者，而是自行假設並繼續執行，導致最終產出與需求不符。
2. AI 傾向以複雜的架構解決可以用簡單方式處理的問題，引入多餘的抽象層與不必要的功能。
3. AI 在修改指定程式碼時，常順手「整理」周邊不相關的程式碼、格式或註解，造成難以追蹤的副作用。

總歸來說，以上3個問題的共通點，是AI在沒有明確規範的情況下，會自行填補空白，但內容並非是使用者想要的。

一篇抱怨文，意外變成全球12萬人都在用的解法

軟體工程師Forrest Chang在讀到卡帕西的貼文後，將這3個問題整理成4條具體的行為規則，以單一純文字檔案的形式發佈於開源平台 GitHub。

這份檔案名為「CLAUDE.md」，是一份放置於程式專案根目錄的純文字說明書，用途是告訴 AI 助理在協作過程中應當遵守哪些行為準則。

其運作原理類似於企業為新進員工準備的工作守則，規定AI在面對模糊指示、遇到衝突情境，或需要自行判斷時，必須依循哪些原則行事。

免費報名　別讓轉型死於 10% 的試錯！拆解「2026 AI決策領導力課程」，揭秘中高階主管如何靠算力佈局奪回主導權

這份 65 行的檔案在 GitHub 上一天內獲得將近 6,000 人收藏，兩週突破 6 萬，目前已超過 12 萬，成為 2026 年成長最快的單一檔案開源專案。

這4條規則的核心要點分別為：

規則1：寫程式前先思考

實作前務必釐清所有假設與模糊地帶，絕不盲目猜測或替使用者做決定；若有更簡單的解法應主動提出，遇到任何不清楚的地方必須立刻停下來發問。

規則2：簡單至上

只用最少的程式碼解決當下問題，嚴格拒絕任何過度工程化、推測未來需求的功能或不必要的抽象層，確保產出符合資深工程師眼中的「精簡」標準。

規則3：手術式修改

只精準更動與需求直接相關的範圍，絕對不去「順手改善」或重構旁邊未損壞的程式碼及排版，並且只負責清理因本次修改才變成無用的變數或引入。

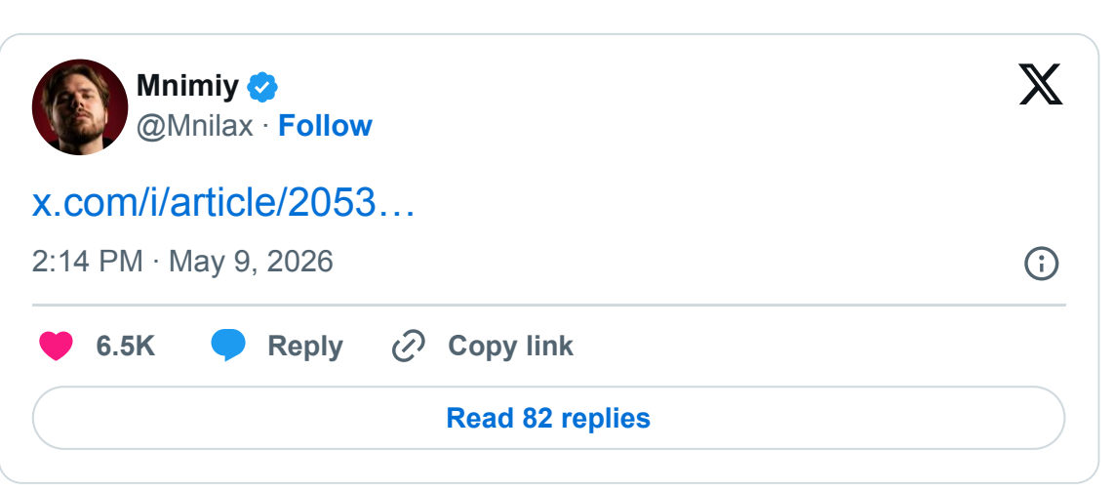
規則4：目標導向執行

將任務轉化為「可被驗證的具體目標」（例如：寫出測試並讓它通過），為多步驟任務建立帶有檢查點的計畫，讓 AI 能基於明確的成功標準自主迭代到完成。

從4條擴增到12條：新的問題在哪裡？

原始4條規則雖然有效，但資深AI工程師Mnimiy在進行系統性測試後發現，它們主要針對的是「單次對話中的寫程式錯誤」，但在面對2026年更複雜的多步驟AI代理協作（Agent-orchestration）與大型專案時，會暴露出以下幾個關鍵漏洞：

1. **無法應付長時間運作的多步驟任務**：原規則預設為單次互動，缺乏 Token 預算與進度檢查點。這導致 AI 在長任務中容易迷失方向、悄悄把錯誤疊加；同時也沒限制 AI 的職責，讓它越界去處理 API 路由等本該由純程式碼執行的確定性邏輯，徒增成本與不穩定性。
2. **在多程式碼庫中會引發風格混亂**：原規則要求「配合現有風格」，但當專案中同時存在新舊兩種相衝的寫法時，AI 會試圖將兩者「平均融合」，反而寫出更難除錯的四不像程式碼。
3. **產生為通過而通過的無效測試**：原規則要求 AI 循環執行直到成功，導致 AI 為了交差，寫出毫無業務價值、只為了亮綠燈的淺層測試（例如把數值寫死），掩蓋了真實的邏輯錯誤。
4. **扼殺原型開發的彈性**：「簡單至上」規則嚴禁任何推測性架構與多餘程式碼，這能有效保護正式環境，但對於需要快速搭建框架、探索方向的原型開發階段，反而會讓 AI 綁手綁腳。



Mnimiy
@Mnilax · Follow
x.com/i/article/2053…
2:14 PM · May 9, 2026

Mnimiy針對這4個缺口，歷經六週、橫跨30個專案，新增了8條規則，填補AI代理時代的系統盲點，讓AI的編寫錯誤率從11%降至 3%。

新增的8條規則解決了什麼問題？

以下是Mnimiy提出的8條規則，用來補足前4條規則的複雜 AI 代理協作的死角。

規則5：只讓 AI 做需要判斷力的事

將 AI 用於分類、摘要、草擬等主觀判斷工作，嚴禁讓 AI 去處理狀態碼判斷、API 重試或路由分配。

只要能用傳統程式碼以 if-else 邏輯寫死的確定性任務，就應交由純程式碼執行，避免模型產生隨機且不可靠的決策。

```
## Rule 5 — Use the model only for judgment calls
Use Claude for: classification, drafting, summarization, extraction from unstructured text.
Do NOT use Claude for: routing, retries, status-code handling, deterministic transforms.
If a status code already answers the question, plain code answers the question.
```

規則6：強制設定詞元預算上限

明確規範單一任務（如 4,000 tokens）與單次對話（如 30,000 tokens）的消耗上限。當運算資源即將耗盡時，模型必須主動總結當前進度並要求重新啟動對話。

此舉可防止模型在無法解決的錯誤中陷入無限迴圈，造成不必要的資源與成本浪費。

```
## Rule 6 — Token budgets are not advisory
Per-task budget: 4,000 tokens.
Per-session budget: 30,000 tokens.
If a task is approaching budget, summarize and start fresh. Do not push through.
Surfacing the breach > silently overrunning.
```

規則7：衝突要攤開講，禁止混合寫法

當程式庫中存在兩種相互矛盾的寫法時，AI 傾向「兩種都照顧到」，寫出一個試圖同時滿足兩種規範的程式碼。這比任何一種原始寫法都更難維護。

規則7要求 AI 選擇其中一種（優先選較新或測試較完整的），說明理由，並標記另一種待日後清理。

```
## Rule 7 — Surface conflicts, don't average them
If two existing patterns in the codebase contradict, don't blend them.
Pick one (the more recent / more tested), explain why, and flag the other for cleanup.
"Average" code that satisfies both rules is the worst code.
```

規則8：寫程式前先讀懂周邊程式碼

在新增或修改任何程式碼之前，模型必須先讀取該檔案的輸出 (exports)、直接呼叫者函數以及相關的共用工具程式碼。

不允許模型在未完全理解現有程式碼結構的情況下，以兩者互不相關為由直接寫入新功能。

```
## Rule 8 — Read before you write
Before adding code in a file, read the file's exports, the immediate caller, and any obvious shared utilities.
If you don't understand why existing code is structured the way it is, ask before adding to it.
"Looks orthogonal to me" is the most dangerous phrase in this codebase.
```

規則9：測試要驗證為什麼，不只是有沒有

模型編寫的測試程式碼必須能真實反映商業邏輯的運作。如果一項業務邏輯發生改變，相關的測試卻依然能夠通過（例如模型為了讓測試亮綠燈而將回傳值寫死），即代表該測試無效。

測試的目的是驗證行為為何重要，而非僅驗證程式有在執行。

```
## Rule 9 — Tests verify intent, not just behavior
Every test must encode WHY the behavior matters, not just WHAT it does.
A test like `expect(getUserName()).toBe('John')` is worthless if the function takes a hardcoded ID.
If you can't write a test that would fail when business logic changes, the function is wrong.
```

規則10：多步驟任務每完成一步就要回報

長時間的重構或功能開發橫跨多個步驟，一旦中途出錯，後續步驟可能全部建立在錯誤的基礎上。

規則10要求 AI 在完成每個重要步驟後，主動回報「已完成事項、已驗證事項、剩餘事項」，若模型在執行過程中遺失上下文，無法精確描述當前狀態，就必須立即中止任務並重新釐清，防止錯誤進度持續疊加。

```
## Rule 10 — Checkpoint after every significant step
After completing each step in a multi-step task: summarize what was done, what's verified, what's left.
Don't continue from a state you can't describe back to me.
If you lose track, stop and restate.
```

規則11：遵從現有慣例，不要偷偷引入新風格

每個程式庫都有自己的命名規則、元件寫法與錯誤處理模式。AI 即便認為自己的寫法更好，也不應在沒有告知的情況下引入新風格，因為「兩種風格並存」比任何一種風格單獨使用都更難維護。

無論模型是否認為某種新框架或寫法更優良，只要專案既定採用特定的命名規則或舊有架構，模型就必須完全配合，禁止在未經討論的情況下擅自引入新風格。

```
## Rule 11 — Match the codebase's conventions, even if you disagree
If the codebase uses snake_case and you'd prefer camelCase: snake_case.
If the codebase uses class-based components and you'd prefer hooks: class-based.
Disagreement is a separate conversation. Inside the codebase, conformance > taste.
If you genuinely think the convention is harmful, surface it. Don't fork it silently.
```

規則12：主動揭露錯誤，禁止隱性失敗

這是 8 條新規則中最受關注的一條。規則要求 AI 在任何步驟有疑問、有遺漏，或無法完整驗證結果時，必須明確回報異常，絕對不允許回報「執行完成」或「測試通過」。

必須將所有不確定性或未執行的步驟完整呈現給開發者，確保沒有任何錯誤被默默忽略。

```
## Rule 12 — Fail loud
If you can't be sure something worked, say so explicitly.
"Migration completed" is wrong if 30 records were skipped silently.
"Tests pass" is wrong if you skipped any.
"Feature works" is wrong if you didn't verify the edge case I asked about.
Default to surfacing uncertainty, not hiding it.
```

完整12條規則一次下載

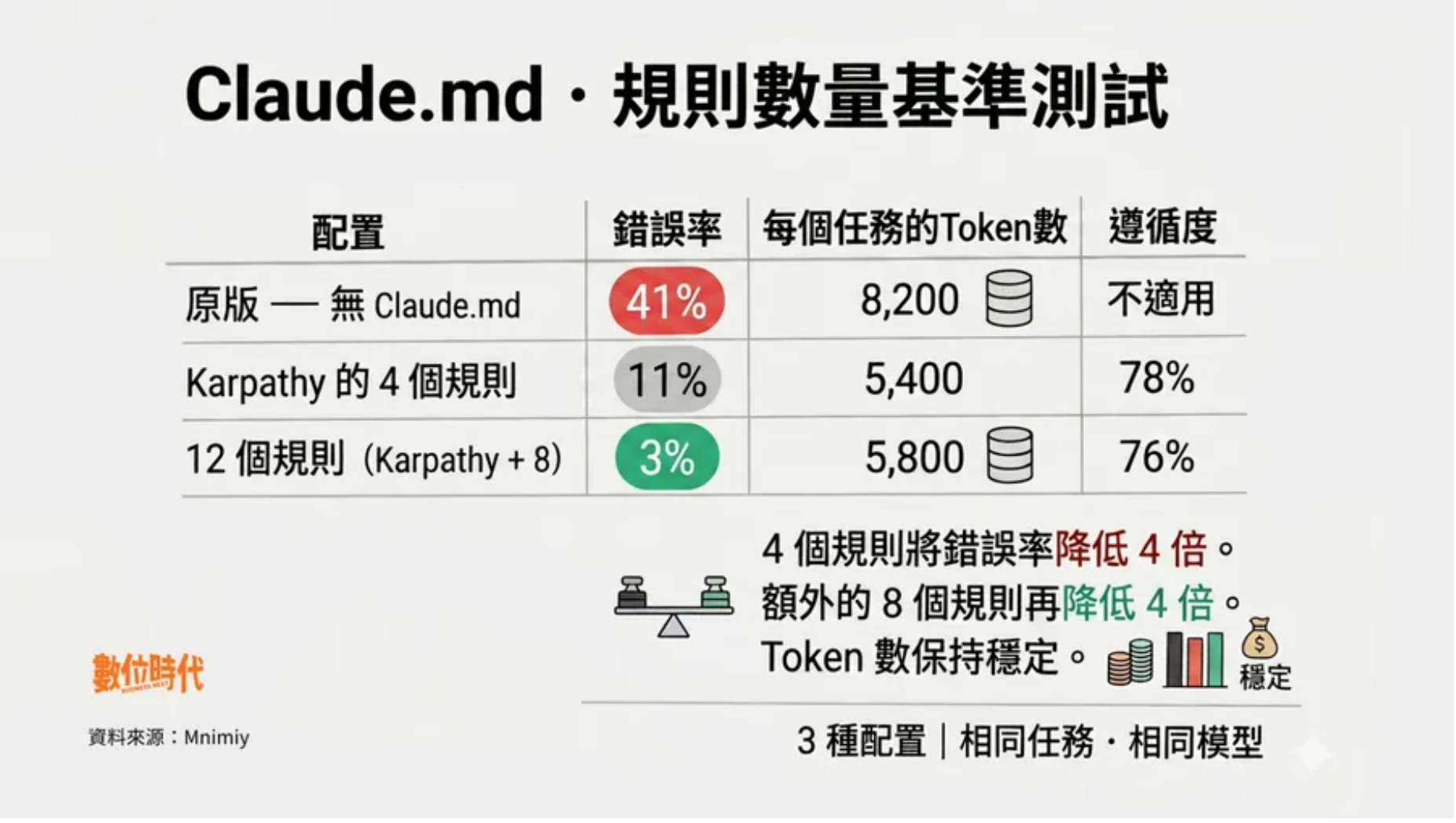
```
# CLAUDE.md — 12-rule template

These rules apply to every task in this project unless explicitly overridden.
Bias: caution over speed on non-trivial work. Use judgment on trivial tasks.

## Rule 1 — Think Before Coding
State assumptions explicitly. If uncertain, ask rather than guess.
Present multiple interpretations when ambiguity exists.
Push back when a simpler approach exists.
Stop when confused. Name what's unclear.

## Rule 2 — Simplicity First
Minimum code that solves the problem. Nothing speculative.
No features beyond what was asked. No abstractions for single-use code.
Test: would a senior engineer say this is overcomplicated? If yes, simplify.

## Rule 3 — Surgical Changes
Touch only what you must. Clean up only your own mess.
Don't "improve" adjacent code, comments, or formatting.
Don't refactor what isn't broken. Match existing style.

## Rule 4 — Goal-Driven Execution
Define success criteria. Loop until verified.
Don't follow steps. Define success and iterate.
Strong success criteria let you loop independently.

## Rule 5 — Use the model only for judgment calls
Use me for: classification, drafting, summarization, extraction.
Do NOT use me for: routing, retries, deterministic transforms.
If code can answer, code answers.

## Rule 6 — Token budgets are not advisory
Per-task: 4,000 tokens. Per-session: 30,000 tokens.
If approaching budget, summarize and start fresh.
Surface the breach. Do not silently overrun.

## Rule 7 — Surface conflicts, don't average them
If two patterns contradict, pick one (more recent / more tested).
Explain why. Flag the other for cleanup.
Don't blend conflicting patterns.

## Rule 8 — Read before you write
Before adding code, read exports, immediate callers, shared utilities.
"Looks orthogonal" is dangerous. If unsure why code is structured a way, ask.

## Rule 9 — Tests verify intent, not just behavior
Tests must encode WHY behavior matters, not just WHAT it does.
A test that can't fail when business logic changes is wrong.

## Rule 10 — Checkpoint after every significant step
Summarize what was done, what's verified, what's left.
Don't continue from a state you can't describe back.
If you lose track, stop and restate.

## Rule 11 — Match the codebase's conventions, even if you disagree
Conformance > taste inside the codebase.
If you genuinely think a convention is harmful, surface it. Don't fork silently.

## Rule 12 — Fail loud
"Completed" is wrong if anything was skipped silently.
"Tests pass" is wrong if any were skipped.
Default to surfacing uncertainty, not hiding it.
```

這份擴充至 12 條規則的 CLAUDE.md 究竟能帶來多大效益？Mnimiy在 30 個不同的程式碼庫中，針對 50 個具代表性的開發任務進行了為期 6 週的嚴格盲測，而最終的數據結果清楚指出了這套「行為契約」的效用。以下整理3大實測亮點：

亮點一：錯誤率呈現兩階段驟降

實測數據展現了錯誤率（指需要開發者手動修正的比例）的兩階段驟降。在沒有任何規則約束的情況下，AI 處理任務的錯誤率高達 41%。當導入 Forrest Chang 整理的 4 條基礎規則後，成功攔截了絕大多數基礎寫碼錯誤，將錯誤率大幅削弱至 11%。

而進一步套用完整的 12 條擴充規則後，原先針對 AI 代理協作與長任務失控的系統盲點被封鎖，使剩餘的錯誤率被極限壓縮至僅剩 3%。



Claude.md・規則數量基準測試

配置	錯誤率	每個任務的Token數	遵循度
原版 — 無 Claude.md	41%	8,200	不適用
Karpathy 的 4 個規則	11%	5,400	78%
12 個規則（Karpathy + 8）	3%	5,800	76%

4 個規則將錯誤率降低 4 倍。
額外的 8 個規則再降低 4 倍。
Token 數保持穩定。
穩定
數位時代
資料來源：Mnimiy
3 種配置｜相同任務・相同模型

圖／ 數位時代

然而，12 條規則在測試中的整體合規率為 76%，意即仍有約四分之一的情況下 AI 不會主動套用規則。此外，規則的效果因任務性質而異，對結構明確的重構任務效果顯著，對高度創意性或探索性的開發工作則效果有限。

亮點二：破除「規則越多越失控」的遵循度迷思

這份測試報告最引人矚目的亮點，在於打破了開發界「規則越多，AI 越不聽話」的迷思。

一般而言，過長的提示詞會導致模型失焦，但實測發現，將規則從 4 條擴充至 12 條後，AI 的指令遵循度（Compliance）僅從 78% 微幅下滑至 76%，意味著開發者幾乎沒有付出額外的遵循度成本，就換取了額外 8% 的錯誤率下降。

亮點三：注意力預算互不衝突

其背後的關鍵在於，這 12 條規則涵蓋了完全不同的觸發情境，原有的 4 條規則處理的是基礎寫碼邏輯，而新增的 8 條規則則是為了應對多步驟任務、預算控管與測試無效等進階痛點，兩者並不會在 AI 處理單一任務時互相爭奪注意力預算。

哪些「常見提示詞」其實是毒藥？

Mnimiy實測發現，網路上流傳的許多寫code提示詞技巧，反而會破壞 AI 的表現：

1. 寫範例不如寫規則

許多人喜歡在提示詞裡舉例，但實測發現，範例非常消耗上下文預算（3 個範例的詞元消耗量相當於約 10 條抽象規則）。更糟的是，AI 會對範例產生「過度擬合 (Over-fit)」，變得不知變通。因此要使用抽象規則，不要用具體範例。

2. 情緒喊話與角色扮演是純雜訊

告訴 AI「請仔細思考」、「你要表現得像個資深工程師」完全沒用。AI 本來就自認是資深工程師，這類無法被驗證的空泛指令，會讓指令遵循度暴跌至 30%。指令必須是具體的動作（例如：明確寫出你的假設），而不是情緒勒索。

3. 依賴特定工具的死指令

規定 AI「永遠使用 ESLINT程式碼檢查工具」是個陷阱，因為一旦專案沒安裝該工具，這條規則就會默默失效。應改用不受工具限制的說法，例如：配合專案強制執行的風格。

Mnimiy也強調，不要盲目套用這 12 條規則，每一條寫進去的規則都必須能回答一個問題：「這能防止我實際遇過的什麼錯誤？」

「一個針對你真實痛點量身打造的 6 條規則，絕對勝過一個塞滿了 6 條你永遠用不到的 12 條規則範本。」Mnimiy說道。

延伸閱讀：AI用錯方式，就算10分鐘也會變笨！研究揭「只問提示、不要答案」一招保住思考力
Claude Code快捷鍵+指令大全！13大類速查不用背，從Ctrl+C到多Agent協作一次整理

資料來源：Mnimiy X

延伸閱讀
- 把壁癌修掉、放進虛擬家具，AI美圖賣屋恐引爆消費糾紛？專家揭2條紅線不能踩
- 執行長都這樣用AI！12位CEO親揭工作術：讓AI當幕僚長、情報蒐集、沙盤推演難關⋯

關鍵字： # Claude

加入數位時代 LINE社群
FUTURE COMMERCE 未來商務　全新改版　新商業解方社群　快去加入！
本網站內容未經允許，不得轉載。

相關新聞　by Taboola
Ce jeu vous empêchera de dormir. Pas de télécharg... Sponsored Links
「AI會取代我嗎？」當焦慮成為職場日常，有人已開... Sponsored Links
À 138€, sans perçage et prêt à l'emploi, les propri... Sponsored Links
「它首先是一台法拉利！」Ferrari首款純電車Luce亮

即時熱門文章
1 「5年後還在不在」才是客戶在乎的！天能綠電手握341億度電，日月光、美光都是它的客戶
2 17組萬用提示詞快收藏！從寫長文到管時間，丟給Claude、ChatGPT、Gemini都能用
3 他實測11款AI做Excel，Claude Cowork勝出！一段提示詞從零生成700條公式，5步驟流程一次學
4 NotebookLM簡報怎麼去AI味？3步驟學會YAML指令，附3款商務模板「複製就能用」
5 「若航空這麼好賺，黃仁勳也能來開」星宇還在燒錢長大，張國煒：想舒服過早就當公子哥，何必來扛
6 ChatGPT Image 2.0指令大全！60組提示詞一次整理，大頭照手機桌布、電玩遊戲像素風、簡報封面全搞定

追蹤我們
掌握新科技・新商業！ 訂《數位時代》1年6期，優惠價1,050元
立即訂閱 >>

AI 時代下「體驗×數據」新行銷玩法　閱讀更多

Future Commerce 未來商務展

客服信箱：service@bnext.com.tw
客服專線：886-2-87716326
服務時間：週一 ~ 週五：09:30~12:00；13:30~17:00

© 2026 Business Next Media Corp. All Rights Reserved. 本網站內容未經允許，不得轉載。

106 台北市大安區光復南路102號9樓

AI 時代下「體驗×數據」新行銷玩法 閱讀更多